



**UNIVERSIDADE DO MINHO
DEPARTAMENTO DE ENGENHARIA E RECURSOS DO MAR**

**CURSO DE LICENCIATURA EM
ENGENHARIA INFORMATICA E SISTEMAS COMPUTACIONAIS**

**TRABALHO DE GRUPO
ANO LETIVO 2025/2026 – 3º ANO**

Tema: Trabalho 2 - Escalabilidade e Balanceamento de Carga em Aplicações

Autor: Aurian da Cruz – Nº 5721, Bruno Soares – Nº 5535

Docente: Frederico Soares

Mindelo, 2026

Índice

1 -	Introdução	1
2 -	Fundamentação teórica	2
2.1 -	Escala Vertical vs Horizontal	2
2.2 -	Load Balancing	3
2.3 -	Replicação de Serviços.....	3
2.4 -	Stateless vs Stateful.....	4
2.5 -	Bottlenecks em Sistemas Distribuídos	4
3 -	Implementação Prática	6
3.1 -	Parte A - Escala no Monólito	6
3.2 -	Parte B - Escala em Microserviços	7
3.3 -	Parte C - Testes de Carga	8
3.4 -	Parte D - Análise Comparativa.....	9
4 -	Diagramas de Arquitectura.....	1
4.1 -	Monolítico	1
4.2 -	API Gateway	2
5 -	Conclusão	3
6 -	Referências Bibliográficas.....	4

1 - INTRODUÇÃO

Este relatório documenta a segunda fase do projecto Sticky Notes Platform, uma aplicação de gestão de notas desenvolvida em Node.js e Express. No primeiro trabalho foi estabelecida a base arquitectural com dois modelos distintos: uma arquitectura monolítica e uma arquitectura de microserviços com API Gateway. O presente trabalho tem como foco a escalabilidade dessas arquiteturas.

A plataforma (distribuída) é composta por dois serviços funcionais — gestão de notas (notes-service) e gestão de pins (pins-service) — comunicando através de um API Gateway. No contexto da escalabilidade, foram implementadas três réplicas do serviço de notas com distribuição de carga via algoritmo round-robin, e o monólito foi paralelizado com o módulo nativo cluster do Node.js. Todos os cenários foram submetidos a testes de carga com um script JavaScript desenvolvido especificamente para o efeito.

2 - FUNDAMENTAÇÃO TEÓRICA

Escalabilidade refere-se à capacidade de um sistema ou processo de lidar com uma quantidade crescente de trabalho de forma eficiente, ou de ser ampliado para acomodar esse crescimento. Num contexto de software, um sistema escalável pode manter ou melhorar o seu desempenho e disponibilidade à medida que a carga de utilizadores ou o volume de dados aumentam.

2.1 - Escala Vertical vs Horizontal

Escala Vertical (Scale Up)

Consiste em aumentar os recursos de uma única máquina, como CPU, memória RAM ou armazenamento. É uma abordagem mais simples, mas tem limites físicos e de custo. Eventualmente, não é possível adicionar mais recursos a uma única máquina.

Escala Horizontal (Scale Out)

Envolve a adição de mais máquinas ou instâncias de serviço para distribuir a carga de trabalho. Em vez de ter um servidor muito potente, tem-se vários servidores menos potentes a trabalhar em conjunto. Esta abordagem oferece maior flexibilidade e resiliência. O The Twelve-Factor App [7] enfatiza a importância de escalar horizontalmente através do modelo de processo.

Dimensão	Escala Vertical (Scale-Up)	Escala Horizontal (Scale-Out)
Mecanismo	Aumentar recursos do mesmo servidor (CPU, RAM)	Adicionar mais instâncias/servidores
Limite físico	Sim — limitado pelo hardware disponível	Não — teoricamente ilimitado
Disponibilidade	SPOF — ponto único de falha	Alta — redundância entre instâncias
Complexidade	Baixa — sem alterações no software	Alta — exige coordenação e load balancing
Custo	Alto por unidade para hardware premium	Mais baixo usando hardware commodity

2.2 - Load Balancing

O balanceamento de carga é o processo de distribuir o tráfego de rede de entrada entre um grupo de servidores de backend, conhecidos como um pool de servidores. O objetivo principal é otimizar o uso de recursos, maximizar o throughput, minimizar o tempo de resposta e evitar a sobrecargas. Conforme descrito por Kleppmann [2], o balanceamento de carga é um padrão fundamental para a construção de sistemas distribuídos escaláveis e resilientes. Existem vários algoritmos de balanceamento de carga, como round-robin, least connections, e IP hash.

Os principais algoritmos de load balancing incluem:

- **Round-Robin:** distribui pedidos sequencialmente por todas as instâncias, em ciclos. Simples e eficaz quando as instâncias têm capacidade homogênea. Implementado neste projecto no API Gateway.[8]
- **Least Connections:** encaminha o pedido para a instância com menos conexões activas. Útil quando os pedidos têm durações variáveis.[8]
- **IP Hash:** determina a instância com base num hash do IP do cliente, garantindo que o mesmo cliente vai sempre para a mesma instância (afinidade de sessão).[8]
- **Weighted Round-Robin:** variante do round-robin onde instâncias com mais capacidade recebem proporcionalmente mais pedidos.[8]

2.3 - Replicação de Serviços

A replicação de serviços envolve a execução de múltiplas cópias idênticas de um serviço para aumentar a disponibilidade e a capacidade. Se uma instância falhar, outras podem continuar a processar requisições. No contexto de microserviços, a replicação é uma prática comum para garantir a resiliência e a escalabilidade de componentes individuais [3].

2.4 - Stateless vs Stateful

Stateless

Um processo stateless não armazena nenhum dado específico da sessão do cliente entre requisições. Cada requisição de um cliente contém todas as informações necessárias para que o servidor a processe. Isso torna os serviços muito mais fáceis de escalar horizontalmente, pois qualquer instância do serviço pode lidar com qualquer requisição a qualquer momento. O The Twelve-Factor App [7] no seu fator VI, "Processes", preconiza que os processos devem ser stateless e "share-nothing", armazenando qualquer estado persistente em serviços de apoio, como bases de dados ou caches distribuídas.

Stateful

Um processo stateful armazena informações sobre a sessão do cliente entre requisições. Isso pode incluir dados de sessão, carrinhos de compras, ou estados de transação. Serviços stateful são mais difíceis de escalar horizontalmente, pois as requisições de um mesmo cliente podem precisar ser direcionadas para a mesma instância do serviço, o que viola o princípio do The Twelve-Factor App [7] e pode levar a problemas de consistência e disponibilidade.

2.5 - Bottlenecks em Sistemas Distribuídos

Bottlenecks são pontos de estrangulamento num sistema que limitam o seu desempenho geral. Em sistemas distribuídos, os bottlenecks podem surgir em várias camadas:

- **Base de Dados:** Uma base de dados mal otimizada ou que não escala adequadamente pode ser um grande bottleneck. O SQLite, por exemplo, embora excelente para aplicações embarcadas ou de uso único, tem limitações inerentes de concorrência quando partilhado entre múltiplas instâncias de aplicação, especialmente sem o modo WAL [5]. Mesmo com o WAL, a documentação do SQLite aponta que a partilha de memória necessária para o wal-index impede o seu uso em sistemas de ficheiros de rede, e que só pode haver um escritor por vez, limitando a concorrência de escrita.
- **Rede:** Latência e largura de banda da rede podem limitar a comunicação entre serviços.
- **CPU/Memória:** Serviços que consomem muitos recursos computacionais podem se tornar bottlenecks se não forem escalados adequadamente.

- **I/O:** Operações de entrada/saída lentas (disco, rede) podem impactar significativamente o desempenho.
- **Serviços Externos:** Dependências de APIs externas ou serviços de terceiros podem introduzir latência e pontos de falha.
- **Algoritmos de Load Balancing:** Um algoritmo ineficiente pode direcionar o tráfego de forma desigual, sobrecarregando algumas instâncias enquanto outras permanecem subutilizadas.

3 - IMPLEMENTAÇÃO PRÁTICA

3.1 - Parte A - Escala no Monólito

O monólito foi escalado horizontalmente utilizando o módulo cluster nativo do Node.js. Este módulo permite criar múltiplos processos (workers) que partilham a mesma porta TCP, com o sistema operativo a distribuir as conexões entre eles. O processo primário gere os workers, reiniciando automaticamente qualquer worker que termine [1].

Ficheiro: monolith-api/cluster.js

```
import cluster from 'cluster';
import os from 'os';

if (cluster.isPrimary) {
  const numCPUs = os.cpus().length;
  console.log(`[primary] PID ${process.pid} - criando ${numCPUs} workers`);

  for (let i = 0; i < numCPUs; i++) {
    cluster.fork(); // cria um worker por CPU
  }

  cluster.on('exit', (worker, code) => {
    console.log(`worker ${worker.process.pid} saiu - reiniciando`);
    cluster.fork(); // tolerância a falhas: reinicia o worker
  });
} else {
  await import('./src/app.js');
  console.log(`[worker ${process.pid}] a escutar`);
}
```

O número de workers é determinado automaticamente pelo número de CPUs lógicas da máquina (os.cpus().length). Numa máquina com 4 núcleos, são criados 4 workers, aumentando o throughput teórico em ~4x para cargas CPU-bound [1]. O evento exit garante resiliência: se um worker falhar, é imediatamente substituído.

3.2 - Parte B - Escala em Microserviços

Na arquitetura de microserviços, o serviço de notas foi replicado em três instâncias independentes, cada uma escutando numa porta diferente. O API Gateway implementa um load balancer round-robin que distribui os pedidos entre as réplicas.

Ficheiro	Porta	Descrição
app.js	3001	Instância principal (original)
appRep1.js	3011	Réplica 1 do notes-service
appRep2.js	3021	Réplica 2 do notes-service
app.js (pins)	3002	Pins-service
app.js (gateway)	3000	API Gateway com load balancer

Round-Robin: routes/notes.js (API Gateway)

```
const NOTES_INSTANCES = [
  'http://localhost:3001',
  'http://localhost:3011',
  'http://localhost:3021',
];

let roundRobinId = 0;

function nextNotesInstance() {
  const url = NOTES_INSTANCES[roundRobinId];
  roundRobinId = (roundRobinId + 1) % NOTES_INSTANCES.length;
  console.log(`[gateway] → ${url} (slot ${roundRobinId})`);
  return url;
}
```

A variável `roundRobinId` é um contador global. A cada chamada `nextNotesInstance()`, o índice avança ciclicamente entre 0, 1 e 2, garantindo que os pedidos são distribuídos para cada instância. O log permite confirmar visualmente a distribuição durante os testes.

3.3 - Parte C - Testes de Carga

O script de teste de carga (loadTest.js) foi desenvolvido em JavaScript nativo (Node.js Fetch API), permitindo simular múltiplos utilizadores concorrentes. O script aceita parâmetros via linha de comandos e mede tempo de resposta, taxa de erros e distribuição de carga por worker.

Script de teste: monolith-api/loadTest.js

```
const TOTAL = parseInt(process.argv[2]) || 128; // total de pedidos

const CONCURRENCY = parseInt(process.argv[3]) || 32; // pedidos simultâneos

async function singleRequest(id) {

  const start = Date.now();
  try {
    const res = await fetch('http://localhost:3000/notes');
    return {
      id,
      status: res.status,
      ms: Date.now() - start,
      worker: res.headers.get('x-worker-pid') // PID worker
    };
  } catch (e) {
    return { id, status: 'ERR', ms: Date.now() - start, worker: null };
  }
}

// Envia pedidos em lotes de CONCURRENCY
for (let i = 0; i < TOTAL; i += CONCURRENCY) {
  const batchSize = Math.min(CONCURRENCY, TOTAL - i);
  const batch = Array.from({ length: batchSize }, (_, j) => singleRequest(i + j));
  const batchRes = await Promise.all(batch);
  results.push(...batchRes);
}
```

O script envia 128 pedidos totais em lotes de 32 simultâneos (por padrão, mas pode ser alterado). O cabeçalho x-worker-pid adicionado pelas respostas do monólito permite rastrear qual worker Node.js respondeu a cada pedido, confirmando a distribuição de carga pelo módulo cluster. As métricas recolhidas são:

Total de pedidos: número total enviados

Taxa de sucesso (OK): pedidos com HTTP 200

Taxa de erro: pedidos com falha ou código de erro

Tempo médio de resposta: média aritmética dos tempos de todos os pedidos (ms)

Distribuição por worker PID: número de pedidos tratados por cada worker

3.4 - Parte D - Análise Comparativa

A tabela seguinte apresenta uma comparação detalhada entre as duas arquitecturas nas dimensões relevantes para a escalabilidade:

Critério	Monólito	Microserviços
Escalabilidade	Horizontal via cluster do Node.js; escala a aplicação completa de uma vez	Escala serviço a serviço; notas escaladas independentemente de pins
Complexidade de Operação	Baixa — um único processo (ou cluster de um único módulo)	Alta — 5 processos a gerir; dependências entre serviços
Performance (latência)	Baixa latência — tudo na mesma memória; sem overhead de rede	Overhead HTTP entre serviços; 2+ chamadas de rede por operação
Resiliência	Falha de um worker é automaticamente recuperada pelo cluster	Falha de 1 réplica não afecta as outras; gateway continua a funcionar
Isolamento de Falhas	Baixo — bug em pins pode afetar todo o monólito	Alto — falha no pins-service não afecta o notes-service
Deploy Independente	Não — qualquer alteração requer redeploy completo	Sim — cada serviço pode ser actualizado individualmente
Gestão de Estado	Simples — base de dados partilhada pelo cluster via FS	Complexa — múltiplas DBs; acessos devem ser consistentes

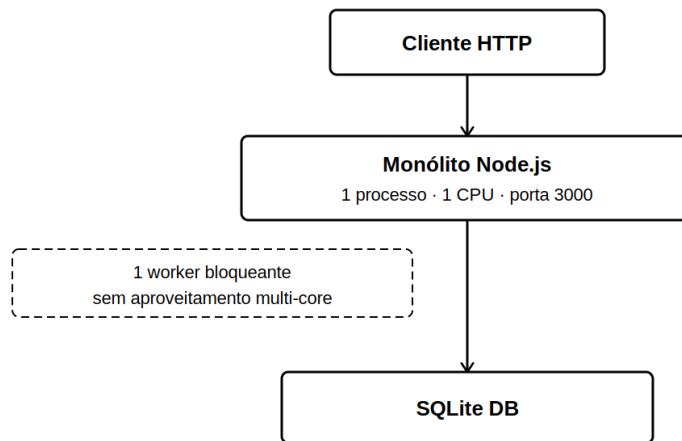
O monólito com cluster apresenta menor complexidade operacional e menor latência por pedido, sendo adequado para aplicações com carga moderada e equipas pequenas.

Os microserviços oferecem melhor escalabilidade granular, resiliência e isolamento, mas ao custo de maior complexidade

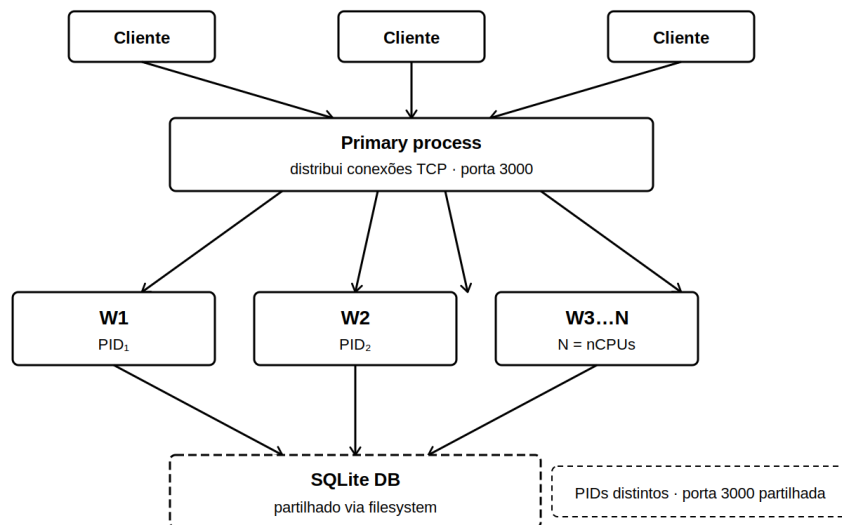
4 - DIAGRAMAS DE ARQUITECTURA

4.1 - Monolítico

Antes

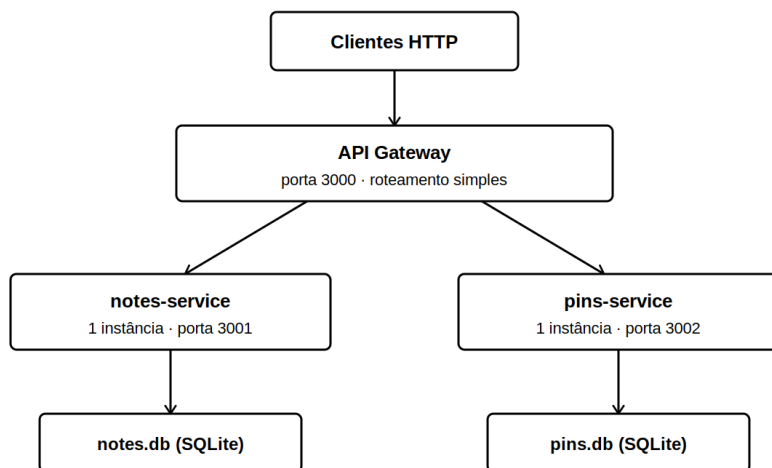


Depois

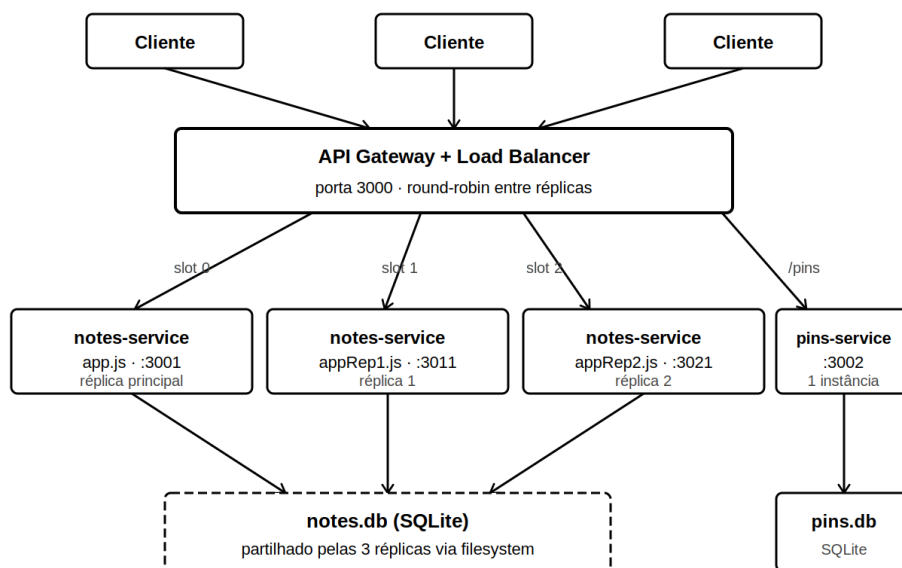


4.2 - API Gateway

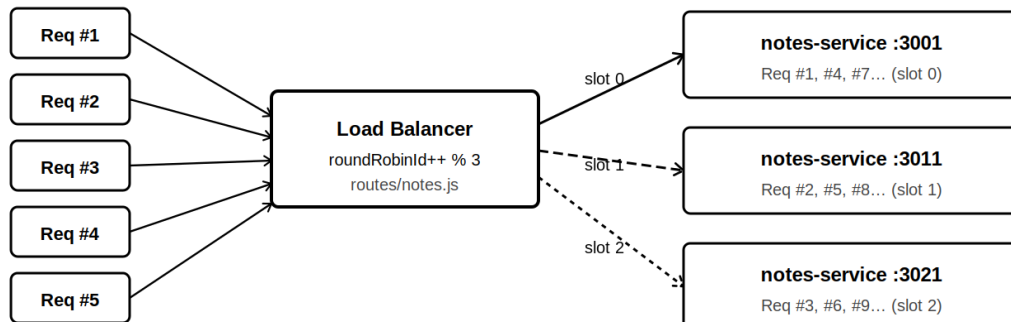
Antes



Depois



Round-Robin



5 - CONCLUSÃO

A evolução da arquitectura base (Trabalho 01) para uma arquitectura escalada permitiu responder directamente às questões centrais do trabalho.

O que acontece quando o número de utilizadores aumenta? Sem escalabilidade, um único processo Node.js fica saturado, o event loop não consegue processar todos os pedidos concorrentes em tempo útil, os tempos de resposta aumentam e eventualmente começam a ocorrer erros. Com o módulo cluster, o trabalho é distribuído por múltiplos workers, cada um com o seu próprio event loop, aumentando significativamente o throughput.

Como escalar os serviços? No monólito, com o módulo cluster do Node.js, criando N workers. Nos microserviços, replicando o serviço mais solicitado (notes-service) e implementando um load balancer no API Gateway para distribuir equitativamente os pedidos pelas três réplicas.

Quais os limites de cada abordagem? O monólito com cluster está limitado pelo número de CPUs da máquina e pela incapacidade de escalar serviços individualmente. Os microserviços superam esse limite, mas introduzem overhead de rede entre serviços, complexidade operacional e o desafio da consistência de dados entre réplicas.

Em ambos os casos, a base de dados SQLite é o principal bottleneck limitando o acerto na comparação entre testes de carga. Em contramedida, a implementação deveria ser substituída por um SGBD cliente-servidor como PostgreSQL, que suporta nativamente múltiplos clientes concorrentes.

6 - REFERÊNCIAS BIBLIOGRÁFICAS

- [1] Node.js Foundation. *Cluster - Node.js Documentation*. <https://nodejs.org/api/cluster.html>, 2024.
- [2] Kleppmann, M.. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [3] Richardson, C.. *Microservices Patterns*. Manning Publications, 2018.
- [4] Newman, S.. *Building Microservices (2nd ed.)*. O'Reilly Media, 2021.
- [5] SQLite Consortium. *SQLite Documentation - Locking And Concurrency*. <https://www.sqlite.org/lockingv3.html>, 2024.
- [6] Express.js Team. *Express.js Documentation*. <https://expressjs.com>, 2024.
- [7] The Twelve-Factor App. (n.d.). The Twelve-Factor App. <https://12factor.net/>
- [8] NGINX. (n.d.). NGINX Load Balancing Guide. <https://www.nginx.com/resources/glossary/load-balancing/>
- [9] autocannon. (n.d.). autocannon (npm). <https://www.npmjs.com/package/autocannon>